Knapsack problem

author: Konrad Basza (konrad.basza@student.put.poznan.pl)
source code: https://github.com/Aviuss/1---knapsack.git

Description of the algorithm

As a base of the algorithm, I used source code from the tutorial (https://ktiml.mff.cuni.cz/~pilat/media/materials/pia/sga.py).

Individuals are represented as a python *set* containing indexes to the global array with cost and weights of knapsack elements. That way, there are no duplicate elements.

Fitness function is the total cost of elements in the *set*. If the weight of the elements is greater than specified its fitness function is set to zero. Setting fitness function to zero could cause some problems with convergence, but because of explicit checks, there are no scenarios where fitness function is set to zero, because elements cannot be added when an individual is full.

Selection is done as tournament selection. Although it can be switched to roulette wheel selection (which performs worse).

Crossover is implemented as a single point crossover with crossover point placed in the middle of each parent. That means that one offspring gets the first half of the first parent, and the second part of the second parent. Because each individual is a *set*, there are no repetitions.

Mutations

- There is *deletion mutation*, where some elements are deleted from the individual. The probability of mutation can be set, as well as the expected number of deleted elements in the individual. Additionally, if there is a stagnation in the evolutionary algorithm, the expected number of deleted elements can be dynamically increased, to more easily get out of local maximum. The moment where a function is deemed stagnant can be set by *IS_FLAT_WINDOW* parameter.

- There was an *addition mutation* where, there was some chance to add some new elements to the individual. I decided to replace it with *permanent addition until full* where to each individual some random elements are added as long as there is some space for it. The change greatly improved performance. Knapsack individuals are generally better when they are fully packed.

Elitism was added to improve a chance of preserving the best individual from parent generation to offspring. Some percentage of the best parents are taken into evaluation with offsprings. From the evaluation top *POP_SIZE* elements are taken. That means that some parents can be taken into the next generation, as long as they are better. Initially, the percentage was set to 100%, which means that all parents, and all siblings are taken into the evaluation. That caused fast convergance, and poorer exploration. That's why the parameter value is low.

The initial population is created randomly. Randomly selected elements are added to the individual as long as they have space for them. That way, each individual has a *set* of random elements that take most of the space.

Running the code

To run the code clone https://github.com/Aviuss/1---knapsack.git repository and install `matplotlib` and `numpy` packages.

Then *python main.py --file test_data/input_100.txt*

File parameters can be changed to load different sets of data.

At the end of the *main.py* source code there is a structure to define evolutionary algorithms parameters.

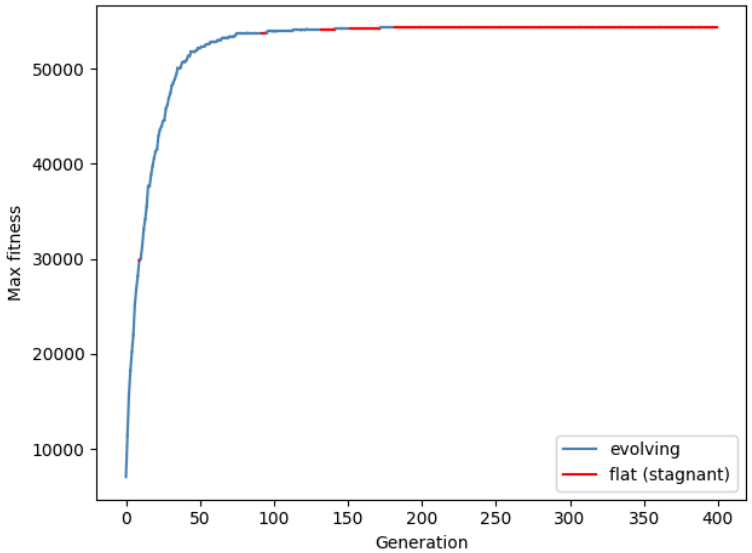
```
def mut_del_elem_prob_func(ind_size, ev_ctx):
        if ev_ctx['current_variable_is_function_flat']:
            if random.random() < 0.1:
                return 50/ind_size
            return 3/ind_size
        return 1/ind_size

ev_ctx = { # evolutionary context
   "MAX_GEN": 400,
```

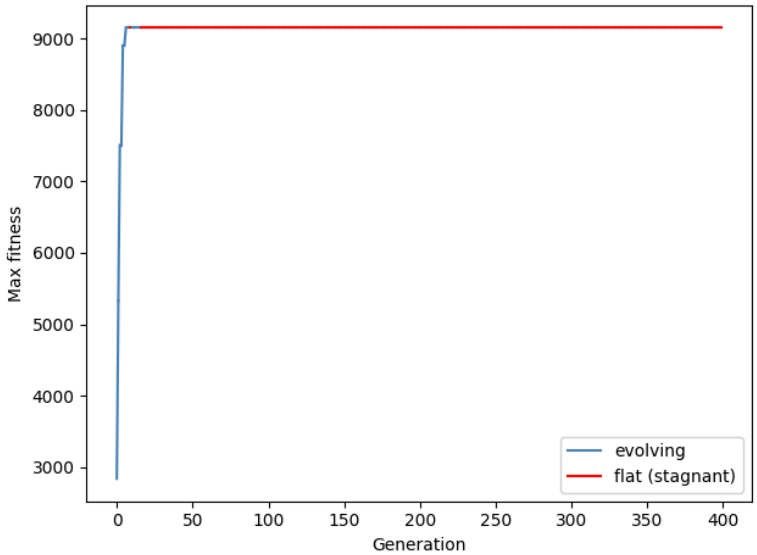
```
    "POP_SIZE": 100,
    "CX_PROB": 0.95,
    "MUT_DEL_PROB": 0.95,
    "MUT_DEL_ELEMENT_PROB": mut_del_elem_prob_func,
    "PERCT_OF_PARENTS_INTO_NEXT_POPULATION": 0.07,
    "W": W,
    "list_price_weight": list_price_weight,
    "n": len(list_price_weight),
    "IS_FLAT_WINDOW": 10,
    "current_variable_is_function_flat": False,
    "TOURNAMENT_SELECTION": 4 # If set to None -> roulete selection
}
```

Best solutions

Those best solutions were obtained using parameters shown in the last chapter.



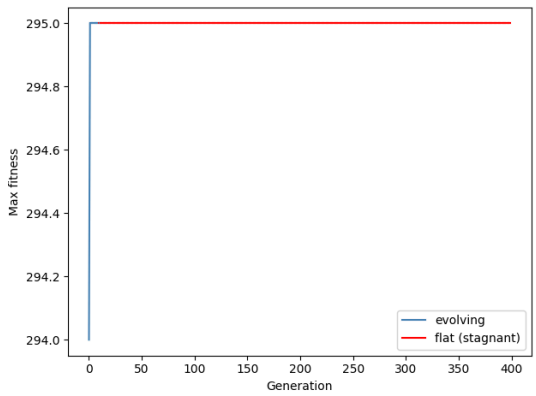
The best value was = 54313.

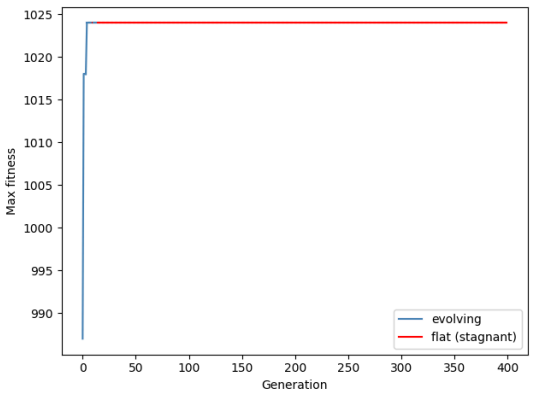


The best value was = 9147.

In both solutions, evolutionary algorithms converged in the beginning. The aggressive deletion was performed during the stagnation period. *Deletion mutation* was set to 50 elements, 1 out of 10 times, and to 3 elements, 9 out of 10 times. After the deletion, individuals are filled with random data. As seen in the first example, when stagnation was detected, aggressive mutation pushed the results out of the local maximum three times. It is hard to estimate how useful aggressive mutation is, because the algorithm rarely goes into stagnation, unless near optimal solution. Probably, it moves the evolutionary algorithm in a new direction faster. Without aggressive mutation, the function behaves quite similarly.

The algorithm also passed *debug_10* and *debug_20* solutions.

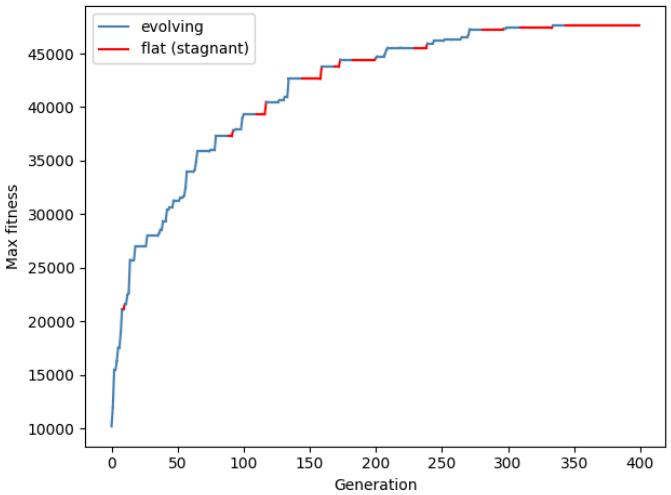




Experimentation

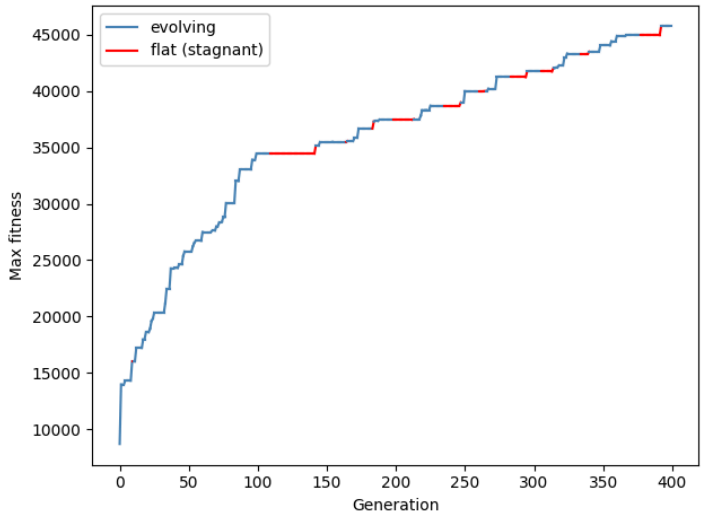
Unless specified otherwise, the evolutionary algorithm configuration is the same as the one in the *Running the code* chapter.

1. Aggresive mutation always. When *deletion mutation* is set to expected 10 elements per individual (which has 100 elements), regardless if the function is flat or not, this is the result:



Because of the constant deletion, (about 10% of each individual is deleted per mutation), the algorithm has a problem with convergence, because it loses data fast.

2. Low probability of crossover and mutation. When both parameters are set to 1%:

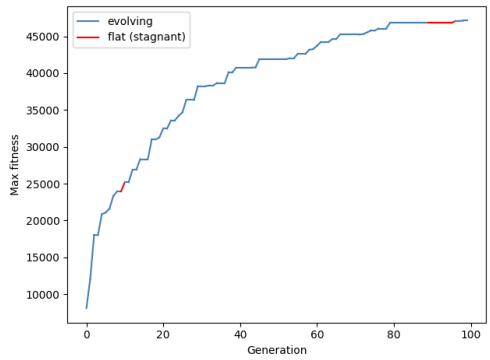


algorithm slowly explores solutions.

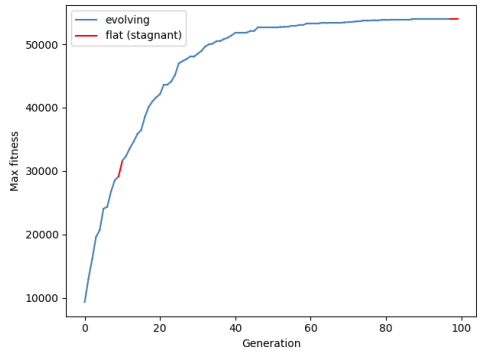
Even when crossover and mutation is higher, like 50% each, it is still worse than, when it's like 95%.

3. Selection method. Tournament selection with *k = 4* vs Roulette wheel selection. *MAX_GEN* was set to 100.

Roulette wheel selection:



Tournament selection:



Running the algorithm multiple times, roulette wheel selection averaged 47123 score. On the other hand, tournament selection averaged 53961.